

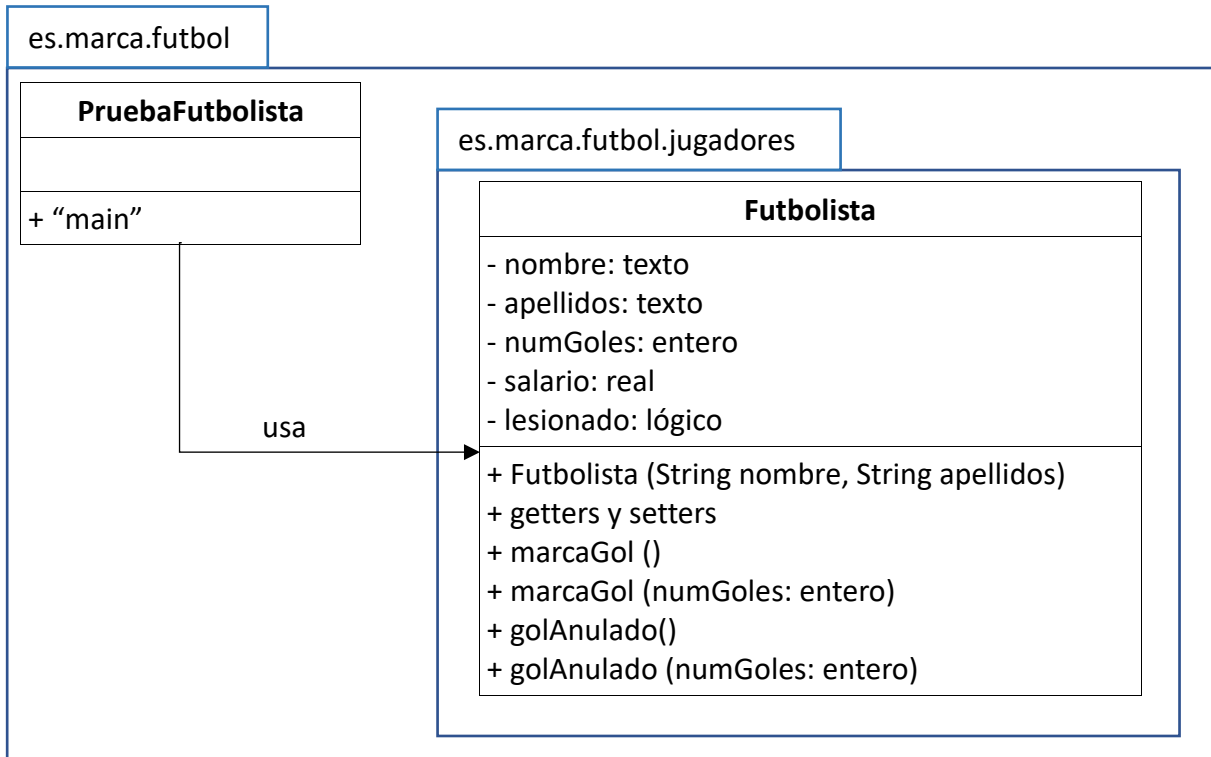
CUESTIONES Y EJERCICIOS

1. Teclea la siguiente clase de modo aplicando las técnicas de encapsulamiento y la sobrecarga de constructores:

Ordenador
- marca: texto - modelo: texto - anioFabricacion: entero - encendido: lógico
+ Ordenador() + Ordenador(marca: texto, modelo: texto) + Ordenador(marca: texto, modelo: texto, anio: entero) + imprimeEstado() + getters y setters

A continuación, crea una clase *PruebaOrdenador* con un método *main* en la que crees tres objetos *Ordenador* utilizando un constructor distinto cada vez. Una vez creados los tres objetos, debes utilizar el método *imprimeEstado()* de cada uno de ellos que mostrará el valor de las propiedades del objeto.

2. Escribe la clase *Futbolista* y pruébala con una clase *PruebaFutbolista* siguiendo el siguiente diagrama:



Debes atender, además, a las siguientes restricciones:

- **marcaGol()** incrementará en uno la cuenta de goles del futbolista.
- **marcaGol (numGoles)** incrementará la cuenta de goles del futbolista añadiendo los que se reciben como parámetro.
- **golAnulado ()** se restará un gol a la cuenta de goles del futbolista
- **golAnulado (numGoles)** realizará la operación de decrementar el número de goles restando los que se reciben como parámetro.

El objeto tendrá que “defenderse” ante los malos usos y evitar que se den situaciones incoherentes (salario o número de goles negativos, que un jugador lesionado marque goles...) añadiendo las instrucciones necesarias para garantizarlo.

La clase **PruebaFutbolista** debe crear un objeto **Futbolista** y probar los métodos sobrecargados.

3. Modifica el ejercicio 1 para que ahora los constructores incluyan asertos que permitan parar nuestra aplicación cuando el año sea menor que el 1990 o bien cuando las cadenas sean nulas o vacías. Modifica el método *main* para probar los asertos.

4. Modifica el ejercicio 2 para que cualquier método que reciba una cadena verifique que no sea nula o esté vacía, parando la ejecución en caso contrario. También se debe controlar que el salario sea un número positivo. Y, por último, hay que verificar cualquier situación anómala o imposible con la cuenta de los goles, deteniendo la ejecución si es necesario. Modifica el método *main* para probar los asertos.
5. Crea un programa que le pida al usuario el número de alumnos/as matriculados de una clase y a continuación crea un array con tantas posiciones como número de matriculados haya. Ahora debemos recorrer el array rellenándolo con notas enteras aleatorias entre 0 y 100. Ordenamos el array y, por último, tenemos que mostrar los siguientes mensajes:

La nota más baja de la clase es XXX

La nota más alta de la clase es XXX

6. Crea una clase Ciudades con un método main que le pida al usuario el nombre de las 10 ciudades que más le gusten y las vaya guardando en un array. A continuación, se debe ordenar y mostrar el contenido del array ordenado.
7. Crea una clase ComparandoBusquedas que tenga un método main que realice lo siguiente:
 - Cree un array de 10000 números enteros y lo rellene con números aleatorios comprendidos entre el 0 y 999999.
 - A continuación, vamos a sacar una copia de este array usando para ello un método de la clase Arrays. Acto de seguido la ordenamos.
 - Ahora le pedimos al usuario que nos diga un número entero para buscarlo en ambos arrays.
 - Pediremos al sistema el número de nanosegundos actual con:
long antes = System.nanoTime();
 - Realizaremos una búsqueda secuencial del número indicado por el usuario en el array desordenado. Cuando termine la búsqueda pediremos al sistema el número de nanosegundos actual con:
long despues = System.nanoTime();
 - Haremos la resta de ambas mediciones y la imprimiremos en pantalla.
 - A continuación, repetiremos los 4 últimos pasos, pero realizando una búsqueda binaria sobre el array ordenado.

Haz distintas pruebas y observa los resultados. La búsqueda binaria debería ser más rápida que la búsqueda secuencial.

8. Reescribe el método *main* del ejercicio 2 (futbolista) para que se cree un objeto Futbolista y se muestre un menú al usuario con las siguientes opciones:

- 0 – Salir
- 1 – Mostrar estado
- 2 – Marcar varios goles
- 3 – Anular varios goles
- 4 – Cambiar nombre y apellidos
- 5 – Cambiar salario
- 6 – Se lesiona
- 7 – Se recupera

La opción de “Mostrar estado” debe mostrar en pantalla el mensaje:

Nombre Apellidos está *activo/lesionado* ha marcado *xx* goles y gana *xxxx.xx* euros.

Cada opción del menú debe llamar a un método auxiliar de *main* que reciba el objeto de la clase Futbolista como parámetro, le pida al usuario información si fuera necesario y realice la operación correspondiente.

9. Escribe una clase llamada *ListaNombres* que contenga una propiedad encapsulada que sea un array capaz de almacenar hasta 20 cadenas de texto.

La clase tiene que ofrecer un método público que permita *añadir un nuevo nombre* a la lista (provocando un fallo del programa si el nombre es nulo o cadena vacía), este nombre deberá añadirse en el primer hueco disponible que se encuentre. En el caso de que no haya hueco disponible se debe imprimir un mensaje de advertencia.

También se debe permitir *borrar un nombre ya existente*, dejando un hueco en la posición en la que se encontrase. En el caso de que no se encuentre el nombre que se quería borrar se debe imprimir un mensaje de advertencia.

Además, se debe permitir *modificar un nombre de la lista*, para ello debes realizar un método que reciba el nombre antiguo y el nombre nuevo. En el caso de que se encuentre el nombre antiguo se reemplazará con el nombre nuevo. Si no se encuentra se mostrará un mensaje de advertencia en la pantalla.

Crea también un método que permita *imprimir la lista de nombres*, pero sin mostrar los “huecos”.

Por último, realiza una clase *PruebaListaNombres* con un método *main* que utilice un menú de usuario para probar un objeto de la clase *ListaNombres* que use métodos estáticos para realizar el código de cada opción del menú.

¿Qué cambios tendrías que hacer en la clase *ListaNombres* para que el tamaño de array fuera un parámetro que recibe el constructor de la clase?

10. Crea una clase *Contacto* que representará los datos de contacto de una persona de modo que se incluyan: nombre, apellidos, teléfono y email. Crea al menos 3 constructores para esta clase. Incluye, además, los getters, setters y asertos que consideres necesarios para garantizar que el estado de los datos de la clase siempre es coherente. Añade también un método que permita imprimir los datos del contacto en pantalla en una sola línea.

Ahora vamos a crear una clase *ListaContactos* que va a funcionar de un modo parecido a la clase *ListaNombres* del ejercicio anterior. Para ello debes saber que:

- Debes crear un constructor que permita especificar el número de contactos que se podrá almacenar en la lista.
- Debes crear un método que permita añadir un nuevo contacto a la lista, siempre que haya algún hueco disponible y que no exista otro contacto previo que tenga el mismo nombre y apellidos. En caso de que no se pueda realizar la operación se mostrará un mensaje de advertencia.
- Debes crear tres métodos sobrecargados que permitan buscar un contacto. La diferencia entre ellos es que uno buscará por el número de teléfono, otro por el email y otro por *nombre+apellidos*. Los métodos devolverán el contacto encontrado o bien *null*, si no se encuentra.
- Debes crear tres métodos sobrecargados que permitan borrar un contacto. La diferencia entre ellos es que uno buscará por el número de teléfono, otro por el email y otro por *nombre+apellidos*. En caso de que no se pueda realizar la operación se mostrará un mensaje de advertencia.
- Debes crear también un método que permita vaciar la lista, eliminando todos los contactos que hubiera almacenados.

Por último, realiza una clase *PruebaListaContactos* con un método *main* que utilice un menú de usuario para probar un objeto de la clase *ListaContactos* que use métodos estáticos para realizar el código de cada opción del menú.

RETO:

Con todo lo que sabes podrías escribir un programa que intente “adivinar” un número del 1 al 100 que el usuario haya pensado. El programa te dirá algo como “¿Tu número es el 40?. Si ha acertado se lo confirmas, el programa se felicita a sí mismo y te ofrece la posibilidad de jugar otra vez o de terminar.

En caso de no haber acertado, entonces te preguntará ¿Tu número es mayor que 40? Y tendrás que responderle con la “verdad”. De este modo la máquina tendrá cada vez un rango menor de números con los que probar.

En el caso de que el usuario “mienta” y el rango de números candidatos de la máquina sea incoherente (por ejemplo, si se le dijo que el número buscado era mayor que 22 y menor que 22) entonces la máquina se quejará diciendo “¡¡Tramposo!! Tu número debe ser el 22, pero me dices que debe ser mayor y menor que 22” y el programa terminará de forma abrupta.